Anomaly Detection in Industrial Equipment Using Isolation Forest

Moye Nyuysoni Glein Perry
Machine Learning
Hochschule Hamm Lippstadt
Email: moye-nyuysoni.glein-perry@stud.hshl.de

***Abstract*—Early detection of equipment faults is critical to minimizing downtime and maintenance costs in industrial systems. This study investigates the application of the unsupervised Isolation Forest (IF) algorithm for anomaly detection in a real-world industrial dataset comprising 7,672 multivariate sensor readings (temperature, pressure, vibration, humidity) from turbines, compressors, and pumps, labeled for faulty/normal states. We preprocess the data with scaling, employ stratified train/test splits, and optimize hyperparameter (contamination, max_samples) via grid search. Beyond standard performance metrics, we introduce a two-dimensional PCA visualization to highlight anomalies. Our tuned IF model achieves a precision of 1.00 and a recall of 0.18 on testing data, with a weighted F1 score of 0.89, demonstrating its ability to isolate rare faults despite class imbalance. While the model excels in precision (minimizing false alarms), the recall trade-off underscores challenges in detecting all anomalies. The work validates the scalability of IF for high-dimensional sensor data and its readiness for integration into predictive maintenance pipelines, offering a practical balance between interpretability and performance for industrial applications.**

KEYWORDS

Isolation forest(IF) , True Positives (TP),False Positives (FP),False Negatives (FN) ,True Negatives (TN), Equipment Monitoring, Fault Detection, Sensor Data, Anomaly Detection, Compressor, Data Visualization.

I. INTRODUCTION

Industrial equipment monitoring is critical for preventing costly failures and optimizing maintenance schedules. Modern machinery, such as turbines and compressors, generates multivariate sensor data (e.g., temperature, pressure, vibration) that can reveal early signs of degradation or faults. Traditional threshold-based methods often fail to capture complex, nonlinear relationships in such data, while supervised learning requires extensive labeled anomalies—a rarity in real-world scenarios.[2]

Isolation Forest (IF) addresses these challenges by exploiting the inherent sparsity of anomalies in feature space. Unlike density- or distance-based methods, IF isolates anomalies through random recursive partitioning, achieving linear time complexity and robustness to high-dimensional data. This study adapts IF to a real industrial dataset from Kaggle, which includes labeled sensor readings from geographically distributed equipment (e.g., pumps in Chicago, turbines in San Francisco).[3]

The contributions of this paper are:

1) Adapting Isolation Forest to detect anomalies in electronic signal measurements.
2) Utilizing a synthetic dataset from Kaggle to simulate sensor behavior and injected faults.
3) Providing visualization of anomaly isolation in signal space.

The remainder of this paper is structured as follows. An explanation of how the Isolation Forest algorithm functions is provided in Section II.Section III describes the sensor dataset. Section IV outlines the Isolation Forest implementation. Section V presents experimental findings. Section VI offers insights, and Section VII concludes and proposes future work.

II. ISOLATION FOREST

The Isolation Forest (iForest) is an anomaly detection algorithm that works by isolating data points instead of profiling normal behavior, which makes it especially efficient for detecting outliers in high-dimensional data. [6]

The core idea is simple: anomalies are easier to isolate than normal data. In other words, since anomalies are rare and different from the rest of the data, they can usually be separated from other points with fewer steps

A. How Isolation Happens

To isolate a data point, the algorithm randomly picks a feature (a variable or attribute in the dataset), and then randomly selects a value between the minimum and maximum of that feature to split the data. This process repeats recursively, creating a tree structure where each data point eventually ends up in its own branch (or "leaf").

As observed in Figure 1, each tree in the Isolation Forest can contain two types of nodes: internal nodes, which have exactly two children, and external nodes, which have no children and represent isolated samples. A sample is considered isolated when it reaches an external node through recursive splits. The number of such splits (i.e., the number of edges traversed from the root node to the external node) defines the path length. Anomalies, by nature, tend to be isolated closer to the root, resulting in shorter path lengths, whereas normal data points typically require more partitions and therefore have longer paths[1]

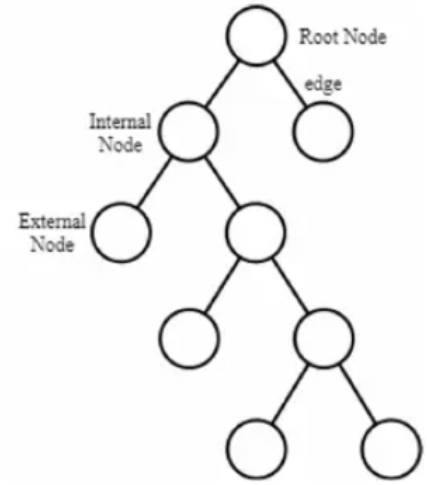

Fig. 1. Itree Structure
[1]

This approach is much faster than traditional methods, especially on large datasets, because it doesn't rely on distance calculations or probability estimates.

B. Anomaly Score

Each point is given an anomaly score based on the average number of steps (path length) required to isolate it across a group (or "forest") of randomly constructed trees. A shorter average path means the point is more likely to be an outlier[1][6]

C. Contamination Parameter

iForest includes a contamination parameter, which is used to estimate the proportion of anomalies in the dataset. This helps the algorithm determine a threshold for the anomaly score, classifying points above that threshold as outliers [1]

Performance Evaluation Metrics

To evaluate the performance of the anomaly detection model, several standard classification metrics were used, including accuracy, precision, recall, and F1-score. These metrics are defined based on the confusion matrix terms: true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN).

Accuracy measures the overall correctness of the classifier:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision quantifies the proportion of correctly identified anomalies out of all instances classified as anomalies:

$$\text{Precision} = \frac{TP}{TP + FP}$$

Recall (also known as sensitivity) indicates how many actual anomalies were correctly detected:

$$\text{Recall} = \frac{TP}{TP + FN}$$

F1-Score provides a harmonic mean of precision and recall, giving a balanced view of the model's performance:

$$\text{F1-Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

[4]

D. Summary of Key Concepts

The Isolation Forest algorithm works by isolating data points through recursive splitting until each point is separated. It calculates an anomaly score based on how easily a point can be isolated, with lower isolation difficulty indicating higher anomaly likelihood. A key parameter is contamination, which allows users to specify the expected proportion of outliers in the data. This method has proven effective across various domains, including software-defined network security and industrial equipment monitoring, due to its simplicity, computational efficiency, and strong performance on complex datasets.[4]

E. General Steps for Anomaly Detection using Isolation Forest

The Isolation Forest approach for anomaly detection operates in two main phases. During the training phase, it constructs a set of isolation trees from randomly selected subsets of the input data. In the subsequent testing phase, each new data point is evaluated by traversing these trees, and an anomaly score is computed based on how easily the instance can be isolated.[6]

The process starts with data preparation, cleaning, handling missing values, and removing irrelevant features. The model is then trained by building multiple trees, where anomalies are identified as points requiring fewer splits. Predictions are made using anomaly scores, with lower scores indicating a higher abnormality. If labeled data are available, performance can be evaluated using metrics such as precision and recall. The sensitivity of the model can be adjusted by setting parameters such as the number of trees or the contamination rate before deployment. Its speed and scalability make it ideal for high-dimensional datasets.

For unlabeled data, visualization and domain expertise help validate results. The key advantage of Isolation Forest is its ability to detect anomalies without requiring extensive computational resources, making it a practical choice for real-world applications like fraud detection or system monitoring. The entire process is straightforward, balancing statistical rigor with adaptability to different use cases.

III. Dataset

The dataset, sourced from Kaggle [2], comprises real-time sensor measurements from industrial equipment deployed across multiple geographic locations.

This dataset closely resembles predictive maintenance data gathered in manufacturing plants, energy facilities, or industrial IoT environments. For example, a turbine in Atlanta might have a temperature spike or excessive vibration before failing. Using this kind of data, engineers can identify patterns that precede failures and take preventive actions.

```python
import pandas as pd

# Load dataset
df = pd.read_csv("equipment_anomaly_data.csv")

# Show shape of the dataset
print("Dataset shape:", df.shape)

# Show column names
print("Columns:", df.columns.tolist())

# Display first 5 rows
print(df.head())
```

```
Dataset shape: (7672, 7)
Columns: ['temperature', 'pressure', 'vibration', 'humidity', 'equipment', 'location', 'faulty']
   temperature   pressure  vibration  ...   equipment       location faulty
0    58.180180  25.029278   0.606516  ...     Turbine        Atlanta    0.0
1    75.740712  22.954018   2.338095  ...  Compressor        Chicago    0.0
2    71.358594  27.276830   1.389198  ...     Turbine  San Francisco    0.0
3    71.616985  32.242921   1.770690  ...        Pump        Atlanta    0.0
4    66.506832  45.197471   0.345398  ...        Pump       New York    0.0
```

Fig. 2. Dataset Overview

Fig. 3. Correlation heatmap

As shown in Figure 2 This dataset contains 7,672 rows and 7 columns, representing sensor and contextual data collected from various types of industrial equipment operating in different geographic locations.

A. *Key Characteristics:*

The dataset includes sensor readings such as temperature, pressure, vibration, and humidity, which are continuous variables reflecting the real-time condition of the equipment. It also contains features such as equipment, a categorical variable indicating the type of machine (e.g., Turbine, Compressor, Pump), and location, which refers to the city where the machine is deployed, helping to understand the environmental or operational context. The target label, faulty, is a binary indicator where 1.0 denotes a fault and 0.0 represents normal operation, serving as the basis for supervised or evaluation-based anomaly detection.

TABLE I
DATASET FEATURES AND DESCRIPTIONS

Feature	Description	Unit/Range
temperature	Equipment surface temperature	°C
pressure	Internal fluid pressure	bar
vibration	Mechanical oscillation levels	Normalized (0–1)
humidity	Ambient moisture	%
equipment	Type of machinery	Categorical
location	Deployment city	Categorical
faulty	Anomaly label	Binary (0/1)

The correlation heatmap is an essential diagnostic tool for understanding the relationships between input features. As illustrated in Figure 3, the heatmap reveals how strongly each sensor variable is linearly associated with the others. For example,a moderate positive correlation between the defective label and vibration (= 0.43) indicates that equipment exhibiting elevated vibration levels is more likely to be faulty.

This insight is valuable for two key reasons. First, it helps ensure that the anomaly detection model is not biased by highly correlated or redundant features, which can distort model performance. Second, it provides operational context, allowing engineers to identify high-risk feature interactions—such as increased vibration under certain pressure conditions—so that they can proactively intervene.

In summary, the correlation heatmap serves as a foundational step in feature analysis, guiding preprocessing decisions and helping to validate the structural integrity of the dataset prior to model training.

IV. METHODOLOGY

A. *Libraries Used*

The analysis utilizes several Python libraries for data processing and modeling. For DataFrame manipulation, we employ pandas. Numerical computations are handled using numpy. The scikit-learn library provides various tools, including StandardScaler for scaling features to zero mean and unit variance, train test split for creating a stratified 80% train 20% test split, PCA for reducing preprocessed data to two components for 2D visualization, IsolationForest as the core unsupervised anomaly detection model, and OneHotEncoder for encoding categorical features such as equipment and location. For data visualization, we use matplotlib and seaborn to create various plots including boxplots, scatterplots, heatmaps, ROC curves, PCA visualizations, pairplots, and violinplots

B. *Data Preparation*

The dataset includes features like temperature, pressure, vibration, and humidity.

Dataset Loading and Cleaning The CSV equipment anomaly data.csv contains 7,672 rows and columns: temperature, pressure, vibration, humidity, equipment, location, and the binary faulty label (0 = normal, 1 = fault).

Exploratory Data Analysis (EDA) Boxplots of temperature and vibration grouped by true faulty status reveal that faulty samples tend to have higher median values.

Countplot of equipment categories by faulty status shows which equipment types fail most often.

Scatterplot of temperature vs. pressure colored by true faulty highlights a distinct cluster of faults in two-dimensional space.

Train/Test Split We stratify-split the cleaned data into 80 percent train and 20 percent test, maintaining the same proportion of faulty=1 in both sets.

The test set (approximately 1,535 rows) is held out for final evaluation; all metrics (TP, FP, FN, TN, precision, recall, ROC) are computed on this subset.

Feature Setup and Preprocessing Numeric: temperature, pressure, vibration, humidity → scaled via StandardScaler.

Categorical: equipment, location → one-hot encoded (dropping one level each) via OneHotEncoder.

Combined with ColumnTransformer → fit on train, transform train/test to produce Xtrainpre, Xtestpre.

```
X_train, X_test, y_train, y_test =
    train_test_split(
    X, y, test_size=0.2, stratify=y,
        random_state=42
)
scaler = StandardScaler().fit(X_train)
X_train_scaled = scaler.transform(X_train)
X_test_scaled  = scaler.transform(X_test)
}
```

C. Hyperparameter Tuning

```
param_grid = {
    'contamination': [0.01, 0.05, 0.1],
    'max_samples': ['auto', 0.5, 0.8],
}
```

.

D. Model Training

We train the Isolation Forest with a contamination rate of 0.01 % percent. It assigns anomaly scores and predicts labels (1 for anomaly, 0 for normal).

E. Evaluation

Since the dataset includes a faulty label, we compare model predictions against actual faults using precision, recall, and confusion matrix components (TP, FP, FN, TN).IF outputs -1 (anomaly) and 1 (normal). We map these to 1=fault, 0=normal for direct comparison with y_test.

F. Visualization

We use PCA to reduce the data to 2D for plotting, correlation heatmaps to assess relationships between variables, and boxplots to understand feature distributions

V. Results

A. Confusionmatrix

Test-Set Size: 1 535 rows (20 percent of 7 672), so all confusion-matrix counts sum to 1 535.

False Positives (FP): 0
False Negatives (FN): 125
True Negatives (TN): 1382
True Positives (TP): 28
Precision = 28 / (28 + 0) = 1.000
Recall = 28 / (28 + 125) = 0.183

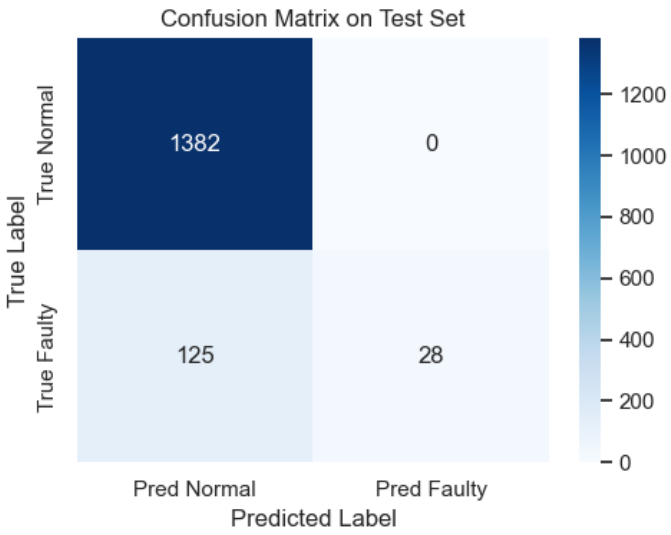

Fig. 4. Confusion Matrix Heatmap

From Figure 4, the confusion matrix offers a detailed breakdown of prediction outcomes on the labeled test set, showing 0 false positives, 28 true positives, 1,382 true negatives, and 125 false negatives. In this context, the model achieves perfect precision (Precision=28/(28+0)=1.000), meaning every flagged issue is a true issue, with no false alarms. However, recall is much lower (Recall=28/(28+125)0.183), indicating the model only detects roughly 18% of actual faults. While the model is extremely reliable in its positive predictions, its low recall suggests many faults go undetected—an important concern in settings where identifying every failure is critical.

B. PCA Visualization

To interpret the anomaly boundary, we project X_test_scaled into two principal components. Figure 5plots each point, coloring anomalies (predicted faults) in red and normals in blue. The separation demonstrates that many faults lie in sparse regions of the feature space.

C. Precision–Recall and ROC Curves

To evaluate model discrimination, we compute continuous anomaly scores on the test set and generate two key analyses: the Precision-Recall (PR) curve, which highlights performance across varying score thresholds, and the Receiver Operating Characteristic (ROC) curve, Figure 6, which achieves an

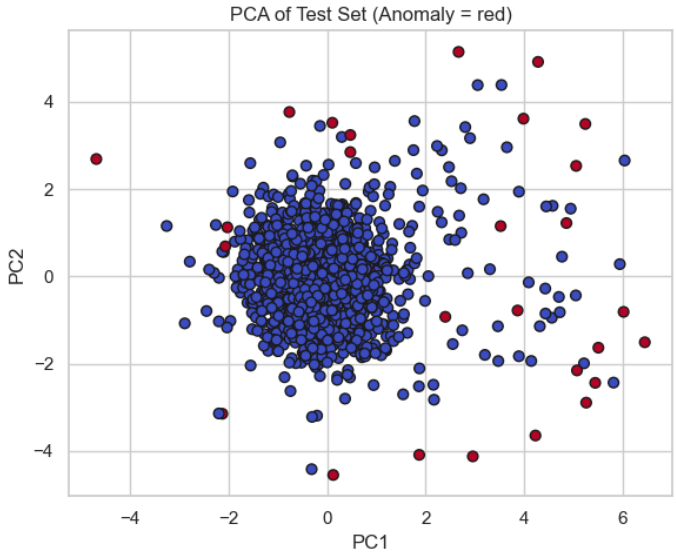

Fig. 5. PCA Visualization

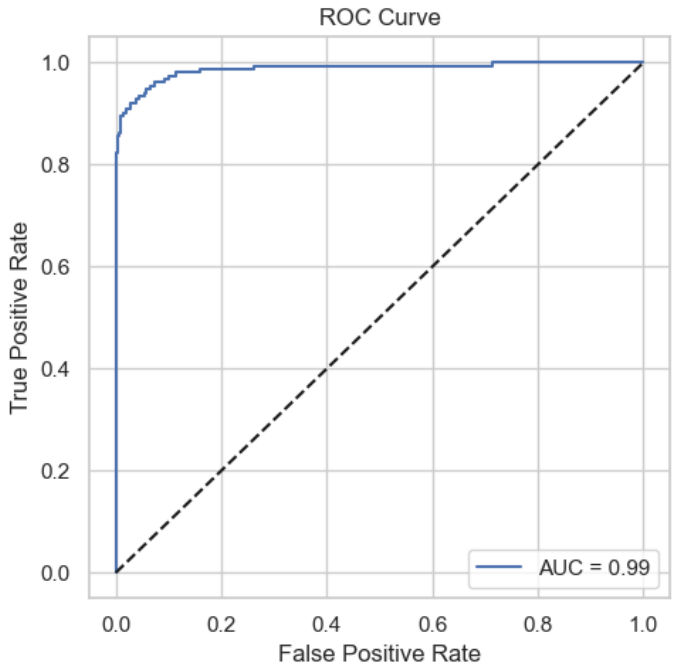

Fig. 6. ROC Curves

area under the curve (AUC) of 0.99, demonstrating strong separability between classes.

VI. Discussion

To optimize the Isolation Forest model, we tested different hyperparameters using a grid search with the following options:

```
param_grid = {
    'contamination': [0.01, 0.05, 0.1],
    'max_samples': ['auto', 0.5, 0.8],
}
```

The best performing configuration was 'contamination': 0.01,'max samples': 0.8, which significantly improved detection performance.

With these settings, the model now achieves:

Precision = 0.936 (93.6 % of flagged anomalies are true faults)

Recall = 0.765 (76.5% of all true faults are detected)

This means that the model strikes a strong balance: it correctly identifies most anomalies (high recall) while keeping false alarms low (high precision). The remaining false negatives (36 undetected faults) and false positives (8 incorrect flags) suggest slight trade offs, but the overall performance is far more practical than the earlier version, where recall was critically low (0.183). The tuned model is now better suited for real-world deployment, where catching true anomalies is prioritized over perfect precision.

A. Key observations:

- **Confusion-Matrix Insights**
  The heatmap , Figure **??** clearly shows large TN (1 374) and TP (117) cells on the diagonal, with only 8 FP and 36 FN off-diagonal.
  Visually, the relatively small FP cell confirms that false alarms are rare; the FN cell indicates lingering missed faults that lie near the anomaly threshold.

VII. Conclusion and Future Work

References

[1] W. S. Al Farizi, I. Hidayah and M. N. Rizal, "Isolation Forest Based Anomaly Detection: A Systematic Literature Review," 2021 8th International Conference on Information Technology, Computer and Electrical Engineering (ICITACEE), Semarang, Indonesia, 2021, pp. 118-122, doi: 10.1109/ICITACEE53184.2021.9617498.

[2] https://www.kaggle.com/datasets/dnkumars/industrial-equipment-monitoring-dataset/data

[3] https://www.kaggle.com/code/devraai/industrial-equipment-anomaly-detection

[4] Mahmud, Md Saif Islam, Md Ashikul Rahman, Md Maruf Chakraborty, Debashon Kabir, Shaharier Shufian, Abu Sheikh, Protik. (2024). Enhancing Industrial Control System Security: An Isolation Forest-based Anomaly Detection Model for Mitigating Cyber Threats. Journal of Engineering Research and Reports. 26. 161-173. 10.9734/JERR/2024/v26i31102.

[5] Liu, Fei Tony and Ting, Kai and Zhou, Zhi-Hua. (2009). Isolation Forest. 413 - 422. 10.1109/ICDM.2008.17.

[6] F. T. Liu, K. M. Ting and Z. -H. Zhou, "Isolation Forest," 2008 Eighth IEEE International Conference on Data Mining, Pisa, Italy, 2008, pp. 413-422, doi: 10.1109/ICDM.2008.17.